MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

Laboratory work n.3

of Cryptography

Kiril Boboc FAF-242

Verified by:
Zaica Maia

Chisinau – 2026

Objective

Implement the Playfair cipher (task 3.1) in Java for messages written in Romanian (an alphabet of 31 letters). The program lets the user choose the operation (encryption or decryption), enter a key of at least 7 characters and a message or a ciphertext, and returns the ciphertext or the decrypted message. The text may contain only the characters 'A'–'Z' and 'a'–'z'; if anything else is entered, the program shows the correct range of characters. Adding the spaces back into the decrypted text is done manually.

1. The Playfair cipher

Playfair encrypts **pairs of letters** (digraphs) instead of single letters. A single plaintext letter is therefore replaced differently depending on its neighbour, so the frequency distribution of the ciphertext is flatter than for a monoalphabetic cipher: the attacker has to analyse 31 × 31 = 961 possible pairs instead of 31 letters. The cipher uses a square matrix built from a keyword and works in three steps: (a) prepare the text, (b) build the encryption matrix, (c) encrypt the pairs of letters.

2. Alphabet and encryption matrix

The Romanian alphabet has 31 letters: A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z. The classic Playfair matrix is 5×5 and merges I and J, but Romanian uses both letters, so the matrix has to be **6×6 = 36 cells**. The 31 letters fill 31 cells, and the 5 free cells are filled with the digits 0, 1, 2, 3, 4. The matrix is filled row by row: first the letters of the key (a repeated letter is written only once, at its first occurrence), then the remaining symbols in alphabetical order.

For the key **MONARCHY** the matrix is the following (the cells taken from the key are highlighted):

M	O	N	A	R	C
H	Y	Ă	Â	B	D
E	F	G	I	Î	J
K	L	P	Q	S	Ș
T	Ț	U	V	W	X
Z	0	1	2	3	4

Figure 1. Encryption matrix for the key MONARCHY

3. Preparing the text

Before encryption the program removes the spaces, converts every letter to uppercase and splits the text into pairs. The following rules are applied:

1. If both letters of a pair would be equal, the letter **X** is inserted between them (the letter **Z** if the repeated letter is X itself).
2. If the final text has an odd number of letters, one extra letter is added at the end (**X**, or **Z** if

the last letter is X). Two equal letters never form a pair.

Message	Prepared text	Pairs
Instruments	`INSTRUMENTSX`	`IN ST RU ME NT SX`
Congress	`CONGRESXSX`	`CO NG RE SX SX`
hello world	`HELXLOWORLDX`	`HE LX LO WO RL DX`

Table 1. Examples of text preparation

4. Encryption and decryption rules

Position of the two letters	Encryption	Decryption
same row	each letter is replaced by the one on its right (the last one by the first of the row)	each letter is replaced by the one on its left (the first one by the last of the row)
same column	each letter is replaced by the one below it (the last one by the first of the column)	each letter is replaced by the one above it (the first one by the last of the column)
different row and column	each letter is replaced by the letter in its own row and in the column of the other letter	exactly the same as for encryption

Table 2. Playfair rules

In the program all three cases use one method, `process(text, shift)`. For encryption `shift = 1` (one step right or down). For decryption one step left or up is needed, that is −1 modulo 6. In Java the operator % keeps the sign of the dividend, so `-1 % 6` gives −1 and would produce an invalid array index. Mathematically −1 mod 6 = 5 (because −1 = (−1)·6 + 5), so the program uses `shift = 5` for decryption: `(c + 5) % 6` moves one column to the left and wraps from the first column to the last one. The value is never negative.

5. Implementation

The program is written in plain Java (only `Scanner`, strings, loops and `char` arrays). The full source code is given in Appendix A. The main parts are:

Method	Purpose
`buildGrid(key)`	removes repeated letters of the key and fills the 6×6 matrix: key first, then the rest of the alphabet and the digits 0–4
`printGrid()`	prints the encryption matrix
`findRow(ch), findCol(ch)`	return the row and the column of a character in the matrix
`prepare(text)`	inserts X (or Z) between equal letters of a pair and pads an odd text
`process(text, shift)`	encrypts (shift = 1) or decrypts (shift = 5) the text pair by pair

Method	Purpose
`onlyLatinLetters(text)`	checks that the key and the message contain only 'A'–'Z'
`onlyAlphabetLetters(text)`	checks that a ciphertext contains only characters of the matrix
`main()`	menu, reading and validating the input, calling the methods above

Table 3. Methods of the program

Input validation.

• The operation must be 1 (encrypt) or 2 (decrypt); otherwise the program asks again.
• The key is read without spaces and converted to uppercase. It must have at least 7 letters and may contain only 'A'–'Z' or 'a'–'z'.
• The message to encrypt may contain only 'A'–'Z' or 'a'–'z' (spaces are removed). For any other character the program shows the correct range: *Use only letters from 'A' to 'Z' or 'a' to 'z'.*
• The ciphertext to decrypt may contain only characters of the matrix and must have an even length.
• The plaintext uses only the Latin letters, but the ciphertext can contain the letters Ă Â Î Ș Ț and the digits 0–4, because these symbols are in the matrix. For example, the pair DX becomes K4 in test 3 below.

6. Tests

Test 1. Key **MONARCHY**, message **Instruments** (matrix in Figure 1). Prepared text: INSTRUMENTSX.

Pair	Case	Result
IN	rectangle	GA
ST	rectangle	KW
RU	rectangle	NW
ME	same column (M, E are in column 1; both move down)	HK
NT	rectangle	MU
SX	rectangle	ȘW

Table 4. Encryption of INSTRUMENTSX

Program output:

```
Choose operation: 1 - Encrypt, 2 - Decrypt
Enter the key (at least 7 letters, only A-Z or a-z):
Encryption matrix:
M O N A R C
H Y Ă Â B D
E F G I Î J
K L P Q S Ș
T Ț U V W X
Z 0 1 2 3 4
Enter the message (only A-Z or a-z):
Prepared message: INSTRUMENTSX
Ciphertext: GAKWNWHKMUȘW
```

Decryption of GAKWNWHKMUŞW with the same key returns `INSTRUMENTSX`. After removing the extra letter X by hand the message is INSTRUMENTS.

Test 2. Key **CRIPTARE** (after removing the repeated R: CRIPTAE), message **Bine ai venit la laborator**.

C	R	I	P	T	A
E	Ă	Â	B	D	F
G	H	Î	J	K	L
M	N	O	Q	S	Ș
Ț	U	V	W	X	Y
Z	0	1	2	3	4

Figure 2. Encryption matrix for the key CRIPTARE

```
Prepared message: BINEAIVENITLALABORATOR
Ciphertext: ÂPMĂCPȚÂORAKFȘPFNICANI
```

This test contains pairs on the same row, for example AI → CP and AT → CA: A is the last letter of the first row, so it is replaced by the first letter, C (wrap-around). Decryption returns BINEAIVENITLALABORATOR, which is read manually as BINE AI VENIT LA LABORATOR.

Test 3. Key **PLAYFAIR**, message **hello world**.

```
Prepared message: HELXLOWORLDX
Ciphertext: ÎGIȚANUSĂPK4
```

The last pair DX is in the same column (D is in row 2, X is in row 5, both in the last column), so X is replaced by the digit 4 from the filler row. The ciphertext with the digit is accepted by the decryption and gives back HELXLOWORLDX.

Test 4. Input validation.

Input	Program response
key abc	The key is too short! It must have at least 7 letters.
message hello1	Wrong characters! Use only letters from 'A' to 'Z' or 'a' to 'z'.
ciphertext AB9C	Wrong characters! The ciphertext can contain only the 31 letters of the alphabet and the digits 0-4.
ciphertext GAK	The ciphertext must have an even number of letters.

Table 5. Invalid input

Conclusion

The Playfair cipher was implemented in Java for the 31-letter Romanian alphabet, with encryption and decryption, a key of at least 7 letters and validation of the input. Because Romanian has both I and J and five letters with diacritics, the classic 5×5 matrix had to be replaced by a 6×6 matrix with five filler digits. The tests show that decryption restores the prepared text exactly, but the extra letters X and Z and the spaces have to be removed and

restored by hand, because the receiver cannot always tell a filler letter from a real one.

Playfair is stronger than a monoalphabetic cipher from the previous laboratory work, because it hides single-letter frequencies: it works on 961 digraphs and the same letter is encrypted differently in different pairs. It is still weak by modern standards. A pair is always encrypted in the same way, reversed pairs give reversed results (if AB becomes XY then BA becomes YX), and the frequencies of digraphs of the language still remain visible in a long text. A longer key makes the analysis harder, but the cipher does not resist statistical cryptanalysis or a known-plaintext attack.

Appendix A. Source code (Playfair.java)

```
import java.util.Scanner;

public class Playfair {

    // Romanian alphabet, 31 letters (written with unicode codes for safety):
    // A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z
    static final String ALPHABET =
            "A\u0102\u00C2BCDEFGHI\u00CEJKLMNOPQRS\u0218T\u021AUVWXYZ";

    // The grid is 6x6 = 36 cells, so we add 5 extra symbols to fill it
    static final String GRID_CHARS = ALPHABET + "01234";

    static char[][] grid = new char[6][6];

    // Build the 6x6 matrix from the key
    static void buildGrid(String key) {
        String used = "";

        // first the key letters (without repeats)
        for (int i = 0; i < key.length(); i++) {
            char c = key.charAt(i);
            if (used.indexOf(c) == -1) {
                used = used + c;
            }
        }

        // then the rest of the alphabet
        for (int i = 0; i < GRID_CHARS.length(); i++) {
            char c = GRID_CHARS.charAt(i);
            if (used.indexOf(c) == -1) {
                used = used + c;
            }
        }

        // fill the matrix row by row
        int k = 0;
        for (int r = 0; r < 6; r++) {
            for (int c = 0; c < 6; c++) {
                grid[r][c] = used.charAt(k);
                k++;
            }
        }
    }

    static void printGrid() {
        System.out.println("Encryption matrix:");
        for (int r = 0; r < 6; r++) {
            for (int c = 0; c < 6; c++) {
                System.out.print(grid[r][c] + " ");
            }
            System.out.println();
        }
    }

    // Find the row of a letter in the matrix
    static int findRow(char ch) {
        for (int r = 0; r < 6; r++) {
            for (int c = 0; c < 6; c++) {
                if (grid[r][c] == ch) {
                    return r;
                }
            }
        }
        return -1;
    }

    // Find the column of a letter in the matrix
    static int findCol(char ch) {
        for (int r = 0; r < 6; r++) {
            for (int c = 0; c < 6; c++) {
                if (grid[r][c] == ch) {
                    return c;
                }
            }
        }
        return -1;
    }

    // Prepare the message: insert X between double letters, pad if odd
    static String prepare(String text) {
```

```
    String result = "";
    for (int i = 0; i < text.length(); i++) {
        char c = text.charAt(i);
        // if we are at the start of a pair and it would be a double letter
        if (result.length() % 2 == 1 && result.charAt(result.length() - 1) == c) {
            if (c == 'X') {
                result = result + 'Z';
            } else {
                result = result + 'X';
            }
        }
        result = result + c;
    }
    // odd length -> add one more letter
    if (result.length() % 2 == 1) {
        if (result.charAt(result.length() - 1) == 'X') {
            result = result + 'Z';
        } else {
            result = result + 'X';
        }
    }
    return result;
}

// shift = 1 for encryption, shift = 5 for decryption (5 is the same as -1 in mod 6)
static String process(String text, int shift) {
    String result = "";
    for (int i = 0; i < text.length(); i = i + 2) {
        char a = text.charAt(i);
        char b = text.charAt(i + 1);

        int r1 = findRow(a);
        int c1 = findCol(a);
        int r2 = findRow(b);
        int c2 = findCol(b);

        if (r1 == r2) {
            // same row
            c1 = (c1 + shift) % 6;
            c2 = (c2 + shift) % 6;
        } else if (c1 == c2) {
            // same column
            r1 = (r1 + shift) % 6;
            r2 = (r2 + shift) % 6;
        } else {
            // rectangle: swap the columns
            int temp = c1;
            c1 = c2;
            c2 = temp;
        }

        result = result + grid[r1][c1] + grid[r2][c2];
    }
    return result;
}

// Only A-Z allowed (used for key and for the message to encrypt)
static boolean onlyLatinLetters(String text) {
    for (int i = 0; i < text.length(); i++) {
        char c = text.charAt(i);
        if (c < 'A' || c > 'Z') {
            return false;
        }
    }
    return true;
}

// Characters of the matrix: 31 letters + digits 0-4 (used for the ciphertext to decrypt)
static boolean onlyAlphabetLetters(String text) {
    for (int i = 0; i < text.length(); i++) {
        if (GRID_CHARS.indexOf(text.charAt(i)) == -1) {
            return false;
        }
    }
    return true;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("=== Playfair cipher (31-letter Romanian alphabet) ===");

    // choose operation
```

```
        int choice = 0;
        while (choice != 1 && choice != 2) {
            System.out.println("Choose operation: 1 - Encrypt, 2 - Decrypt");
            String line = sc.nextLine().trim();
            if (line.equals("1")) {
                choice = 1;
            } else if (line.equals("2")) {
                choice = 2;
            } else {
                System.out.println("Please type 1 or 2.");
            }
        }

        // read key
        String key = "";
        while (true) {
            System.out.println("Enter the key (at least 7 letters, only A-Z or a-z):");
            key = sc.nextLine().replace(" ", "").toUpperCase();
            if (!onlyLatinLetters(key)) {
                System.out.println("Wrong characters! Use only letters from 'A' to 'Z' or 'a' to 'z'.");
            } else if (key.length() < 7) {
                System.out.println("The key is too short! It must have at least 7 letters.");
            } else {
                break;
            }
        }

        buildGrid(key);
        printGrid();

        // read text
        String text = "";
        while (true) {
            if (choice == 1) {
                System.out.println("Enter the message (only A-Z or a-z):");
            } else {
                System.out.println("Enter the ciphertext (31 letters of the alphabet and digits 0-4):");
            }
            text = sc.nextLine().replace(" ", "").toUpperCase();

            if (text.length() == 0) {
                System.out.println("The text is empty!");
            } else if (choice == 1 && !onlyLatinLetters(text)) {
                System.out.println("Wrong characters! Use only letters from 'A' to 'Z' or 'a' to 'z'.");
            } else if (choice == 2 && !onlyAlphabetLetters(text)) {
                System.out.println("Wrong characters! The ciphertext can contain only the 31 letters of the alph
        abet and the digits 0-4.");
            } else if (choice == 2 && text.length() % 2 == 1) {
                System.out.println("The ciphertext must have an even number of letters.");
            } else {
                break;
            }
        }

        if (choice == 1) {
            String prepared = prepare(text);
            System.out.println("Prepared message: " + prepared);
            System.out.println("Ciphertext: " + process(prepared, 1));
        } else {
            System.out.println("Decrypted message: " + process(text, 5));
            System.out.println("(Remove the extra X/Z letters and add the spaces by hand.)");
        }
    }
}
```